

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №11

Интерфейсы

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы.....	3
2 Задание.....	3
3 Описание работы программы	4
4 Результаты работы программы.....	10
5 Вывод	11
Приложение А. Листинг Программы.....	12

1 Цель работы

Цель работы заключается в изучении основ проектирования и реализации программного обеспечения с использованием объектно-ориентированного подхода. В рамках работы необходимо разработать приложение для управления автопарком, которое включает создание классов для автомобилей, их компонентов и управления состоянием. Также акцентируется внимание на использовании интерфейсов и делегатов для обеспечения взаимодействия между объектами, а также на обработке событий и исключений. Работа направлена на закрепление знаний о принципах ООП, а также на практическое применение полученных навыков в разработке программного обеспечения.

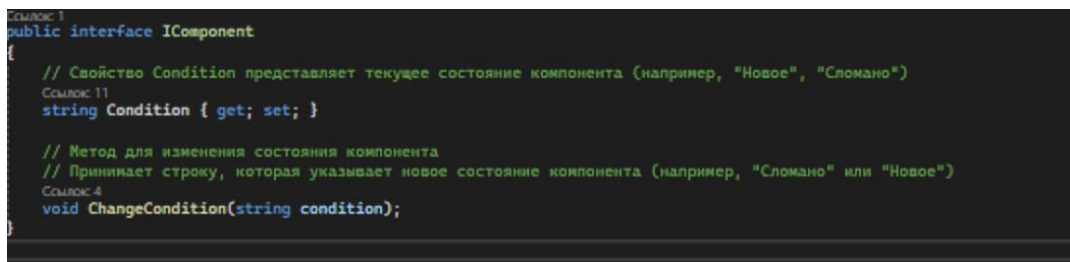
2 Задание

Опишите интерфейсы и реализуйте их функциональность в соответствующих классах. Обязательно продемонстрируйте в приложении и в лабораторной работе взаимодействие объектов различных классов, которые подписаны на эти интерфейсы.

3 Описание работы программы

В программе реализованы интерфейсы и абстрактные классы, которые обеспечивают структуру и функциональность для управления компонентами автомобилей и самими транспортными средствами.

Интерфейс `IComponent` определяет основные свойства и методы, которые должны реализовывать все компоненты автомобиля. Он включает свойство `Condition`, которое отражает текущее состояние компонента, и метод `ChangeCondition`, позволяющий изменять это состояние. Классы, реализующие этот интерфейс, должны следовать заданной структуре, что обеспечивает единообразие в управлении состоянием различных компонентов, таких как колеса и двигатель. На рисунке 1 изображен класс `IComponent`.



```
Рисунок 1
public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента (например, "Новое", "Сломано")
    Ссылка 11
    string Condition { get; set; }

    // Метод для изменения состояния компонента
    // Принимает строку, которая указывает новое состояние компонента (например, "Сломано" или "Новое")
    Ссылка 4
    void ChangeCondition(string condition);
}
```

Рисунок 1 – интерфейс `IComponent`

Абстрактный класс `Component`, который реализует интерфейс `IComponent`, предоставляет общую логику для всех компонентов автомобиля. Он включает реализацию метода `ChangeCondition`, который проверяет допустимые состояния и генерирует событие поломки, если компонент становится "Сломано". Это позволяет другим частям программы подписываться на события поломки и реагировать на них, что улучшает взаимодействие между компонентами. На рисунке 2 изображен класс `Component`.

```

Ссылка 1
public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    Ссылка 9
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    Ссылка 0
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    Ссылка 1
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
    public event EventHandler ComponentBroken;
}

```

Рисунок 2 – класс Component

Интерфейс `IVehicle` определяет методы для управления транспортным средством, такие как `StartEngine` и `StopEngine`. Классы, представляющие различные типы автомобилей, реализуют этот интерфейс, что позволяет им предоставлять конкретные реализации этих методов. Это обеспечивает возможность управления автомобилями через единый интерфейс, что упрощает взаимодействие с различными типами транспортных средств. На рисунке 3 изображен класс `IVehicle`.

```

namespace CarManagementApp
{
    // Интерфейс IVehicle, который определяет методы для управления транспортным средством
    Ссылка 0
    public interface IVehicle
    {
        // Метод для запуска двигателя
        Ссылка 0
        void StartEngine(); // Запуск двигателя

        // Метод для остановки двигателя
        Ссылка 0
        void StopEngine(); // Остановка двигателя
    }
}

```

Рисунок 3 – интерфейс IVehicle

Абстрактный класс `Vehicle`, который реализует интерфейс `IVehicle`,

предоставляет базовую реализацию для всех транспортных средств. Он включает метод `GetInfo`, который возвращает информацию о транспортном средстве, и может быть переопределен в наследующих классах для предоставления более детальной информации. Это позволяет создавать различные типы автомобилей, которые могут иметь свои уникальные характеристики и поведение, сохраняя при этом общую структуру. На рисунке 4 изображен класс `Vehicle`.

```
using System;

Ссылка 0
public abstract class Vehicle
{
    Ссылка 2
    public string Name { get; set; } // Название транспортного средства

    // Базовый метод для получения информации о транспортном средстве.
    // Этот метод будет переопределяться в наследующих классах.
    Ссылка 0
    public virtual string GetInfo(bool showDetailed)
    {
        // Строка с основной информацией о транспортном средстве
        string info = $"Название: {Name}";

        if (showDetailed)
        {
            // Добавление подробной информации, если параметр showDetailed равен true
            info += GetDetailedInfo();
        }

        return info; // Возвращаем информацию о транспортном средстве
    }

    // Этот метод можно переопределить в дочерних классах для предоставления подробной информации
    Ссылка 1
    protected virtual string GetDetailedInfo()
    {
        // Возвращаем информацию о названии транспортного средства
        return $"Название: {Name}";
    }
}
```

Рисунок 4 – класс `Vehicle`

Таким образом, использование интерфейсов и абстрактных классов в программе способствует созданию гибкой и расширяемой архитектуры, позволяя легко добавлять новые компоненты и транспортные средства, а также обеспечивая единообразие в их управлении.

На рисунке 5 изображена диаграмма классов программы.

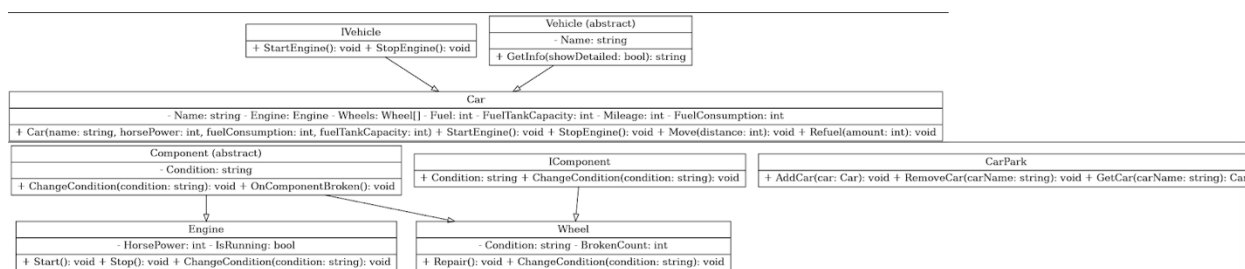


Рисунок 5 – Диаграмма классов

На рисунке 6 представлена структура оформления базового окна Windows Forms.

The screenshot shows a Windows Forms application window titled "Car Management". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main content area is light gray and contains several controls:

- On the left side, there are four text labels with corresponding input fields:
 - "Введите имя авто:" followed by a text box.
 - "Введите мощность авто:" followed by a text box.
 - "Введите расход авто:" followed by a text box.
 - "Введите бак авто:" followed by a text box.
- Below these input fields are two buttons: "Создать авто" and "Удалить авто".
- Below the buttons is a large empty rectangular box.
- At the bottom left, there is a checkbox labeled "Подробно" and a button labeled "Показать".
- On the right side, there are four buttons stacked vertically: "Заправка", "Передвижение", "Запустить", and "Выключить".
- Next to the "Передвижение" button is a checkbox labeled "Высокая скорость".
- At the bottom right, there is a button labeled "Заменить".

Рисунок 6 – Оформление окна Windows Forms

4 Результаты работы программы с примерами

На рисунке 7 представлен результат работы программы показа работы интерфейсов компонента.

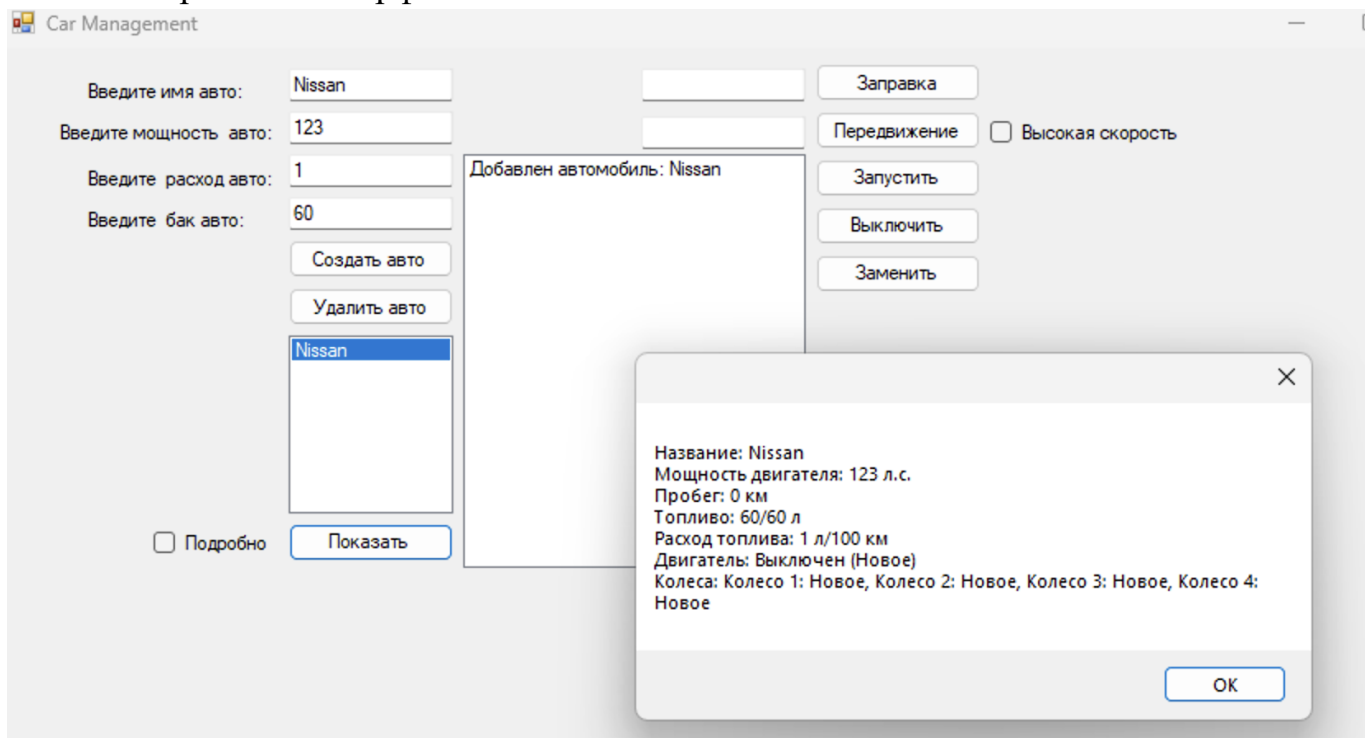


Рисунок 7 – Результат работы программы

5 Вывод

В заключение, разработанная программа демонстрирует эффективное использование интерфейсов и абстрактных классов для создания гибкой и расширяемой архитектуры управления автомобилями и их компонентами. Интерфейс `IComponent` и абстрактный класс `Component` обеспечивают единообразие в управлении состоянием различных автомобильных компонентов, таких как колеса и двигатель, позволяя легко отслеживать их состояние и реагировать на поломки.

Интерфейс `IVehicle` и абстрактный класс `Vehicle` предоставляют основу для управления транспортными средствами, позволяя реализовать различные типы автомобилей с уникальными характеристиками и поведением, сохраняя при этом общую структуру. Это упрощает взаимодействие с автомобилями и делает код более модульным и удобным для дальнейшего расширения.

Таким образом, использование данных принципов объектно-ориентированного программирования способствует созданию надежного и поддерживаемого программного обеспечения, что является важным аспектом в разработке современных приложений.

Приложение А. Листинг Программы

Файл «Component»

```
using System;

public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
```

Файл «IComponent»

```
using System;

public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента (например, "Новое", "Сломано")
    string Condition { get; set; }

    // Метод для изменения состояния компонента
    // Принимает строку, которая указывает новое состояние компонента (например, "Сломано" или "Новое")
    void ChangeCondition(string condition);
}
```

Файл «Info»

```
using System.Linq;

namespace CarManagementApp
{
    public sealed class Info : Car
    {

```

```

// Конструктор для класса Info, который вызывает конструктор базового класса
public Info(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
    : base(name, horsepower, fuelConsumption, fuelTankCapacity)
{
}

// Переопределяем метод получения информации о машине
public override string GetCarInfo(bool showDetailed = false)
{
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

    if (showDetailed)
    {
        info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
        for (int i = 0; i < Wheels.Length; i++)
        {
            info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
        }
    }

    return info;
}
}
}

```